

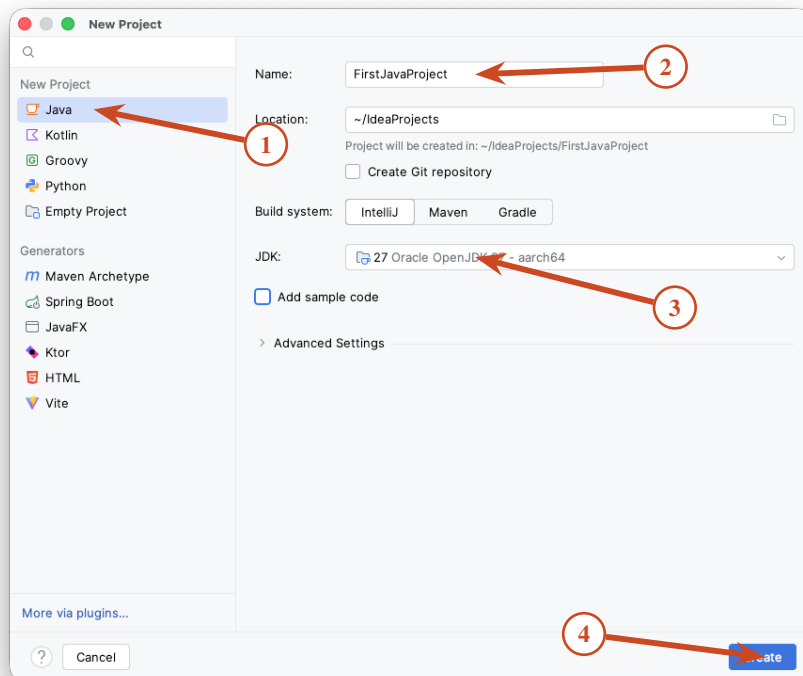
Algorithmes et Pensée Computationnelle

Premiers pas avec Java dans IntelliJ IDEA

NB : Il est recommandé de commencer avec le tutoriel sur Python avant ce tutoriel. La semaine dernière, vous avez appris à créer et à exécuter des programmes depuis la ligne de commande, notamment en leur passant des arguments. Cette semaine, vous allez voir comment réaliser ces mêmes tâches directement dans IntelliJ IDEA.

1 Créer un projet Java dans IntelliJ IDEA

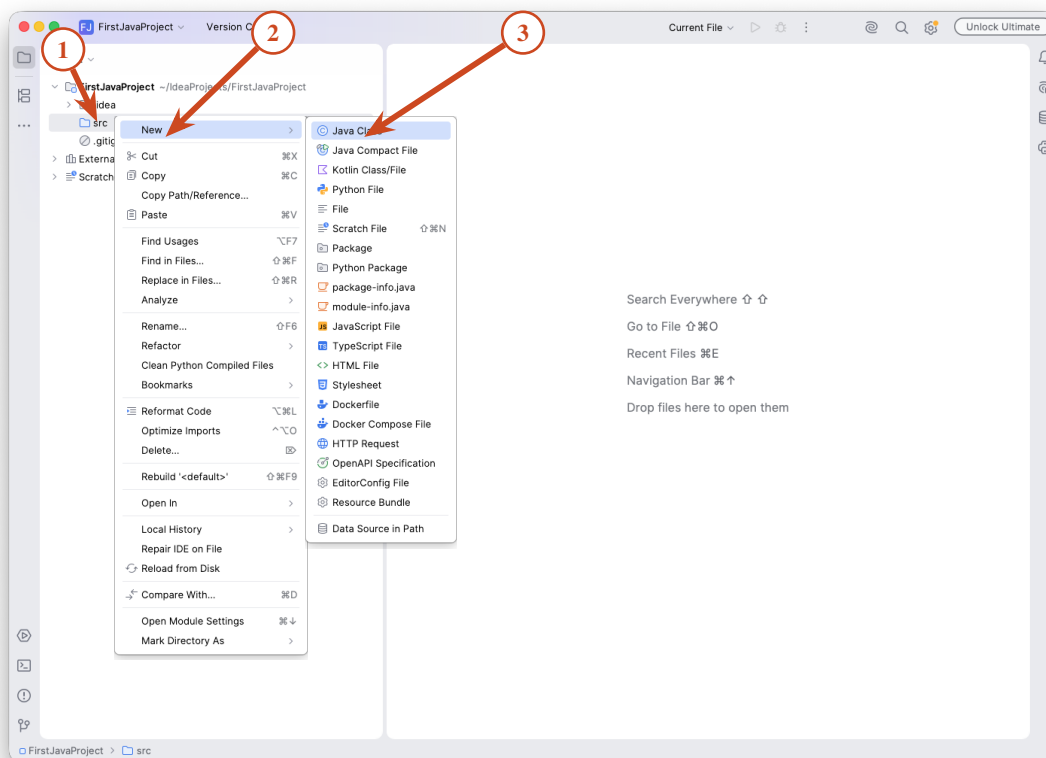
Sur l'écran d'accueil, sélectionnez **New Project**. Si un projet est déjà ouvert, utilisez **File > New > Project**. Dans la fenêtre de création, choisissez **Java** (1), donnez un nom et un emplacement au projet (2), laissez **IntelliJ** comme système de construction, puis sélectionnez un **JDK** installé (3). Le numéro de version et les chemins affichés peuvent être différents sur votre machine. Laissez **Add sample code** décoché, comme sur la capture, et cliquez sur **Create** (4).



Choisissez Java, nommez le projet, vérifiez le JDK et créez le projet.

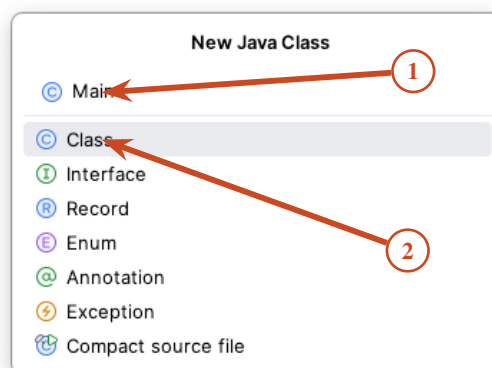
Créer la classe Main

Dans le panneau **Project**, faites un clic droit sur le dossier `src` (1). Dans le menu contextuel, choisissez **New** (2), puis **Java Class** (3). Les deux menus ci-dessous sont replacés sur la fenêtre du projet dans leur ordre de superposition.



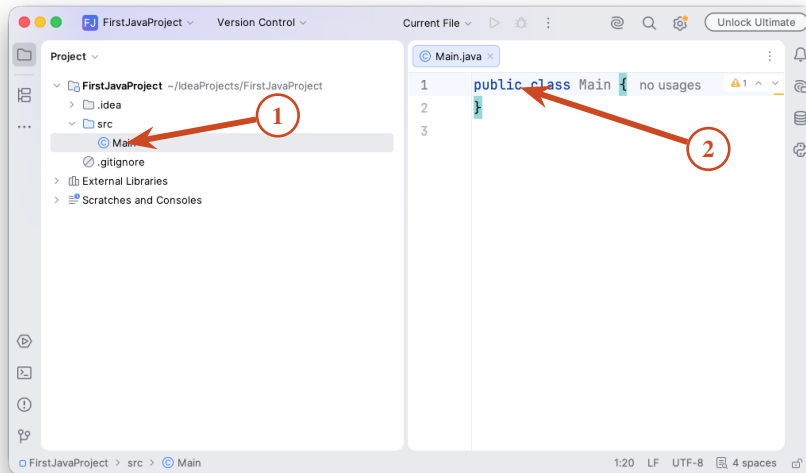
*Clic droit sur `src`, puis **New** et **Java Class**.*

Saisissez `Main` comme nom (1), conservez le type **Class** (2), puis appuyez sur Entrée. IntelliJ crée le fichier `Main.java` dans le dossier `src`.



*Nommez la classe `Main` et laissez **Class** sélectionné.*

Le nouveau fichier apparaît dans le panneau **Project** (1) et s'ouvre dans l'éditeur (2). IntelliJ a déjà écrit la déclaration de la classe et ses accolades. Le code que vous ajouterez doit être placé entre ces deux accolades.



Le fichier Main.java et le corps encore vide de sa classe.

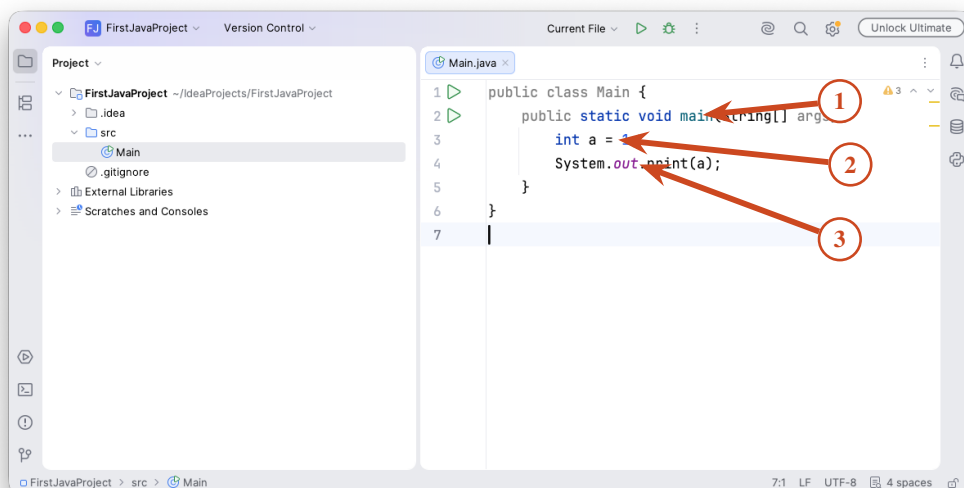
2 Créer des variables

Pour lancer une application Java, ajoutez dans la classe la méthode ci-dessous (1). Elle constitue le point d'entrée du programme.

```
public static void main(String[] args)
```

Dans ses accolades, écrivez `int a = 1;` (2). Le mot `int` indique que `a` contient un nombre entier; le signe `=` lui attribue la valeur 1. Terminez chaque instruction par un point-virgule. La ligne suivante (3) affiche la valeur de `a` :

```
System.out.print(a);
```



La méthode main, la déclaration d'une variable entière et son affichage.

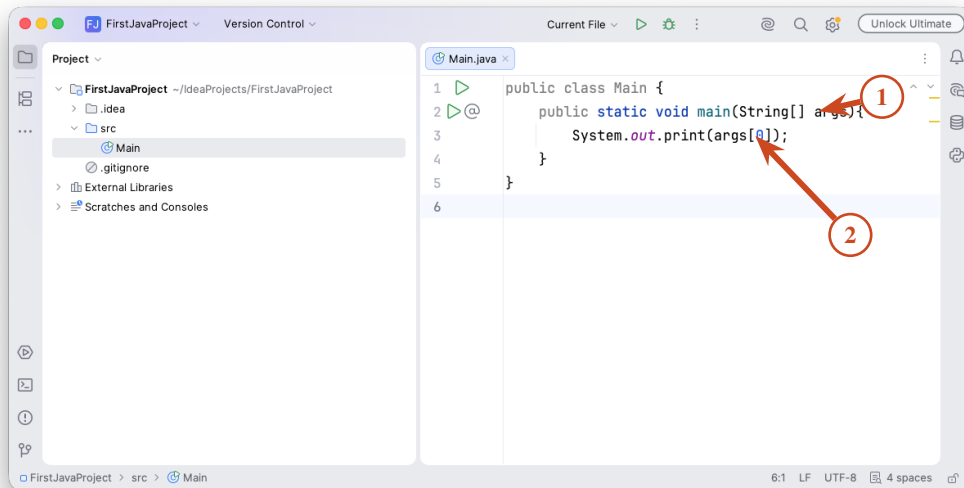
Pour essayer d'autres types, écrivez `String message = "Bonjour";` pour du texte. La déclaration `boolean actif = true;` représente une valeur vraie ou fausse. Placez ces lignes à l'intérieur de `main`. Cliquez sur le triangle vert à côté de `main`, puis choisissez **Run 'Main.main()'**. Le panneau **Run** affiche 1 pour le programme de la capture.

3 Les arguments de la ligne de commande

Les arguments permettent de transmettre des valeurs au programme lors de son lancement. La méthode `main` les reçoit dans le tableau `String[] args` (1). Remplacez les deux instructions de l'exemple précédent par la ligne suivante (2) :

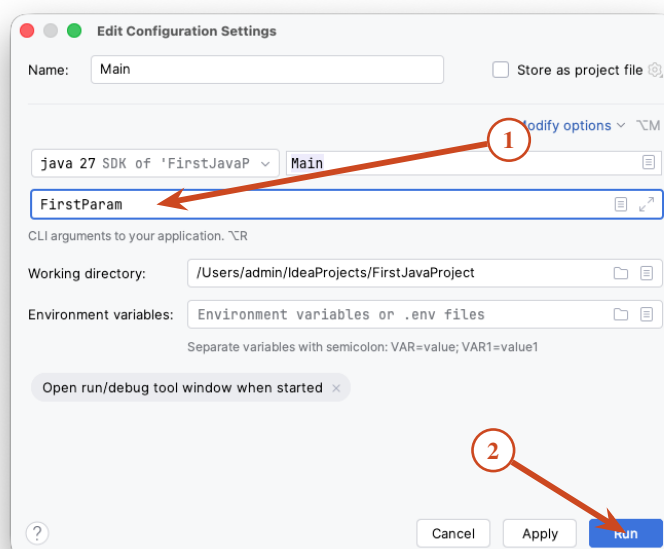
```
System.out.print(args[0]);
```

En Java, `args[0]` est le premier argument fourni ; `args[1]` serait le deuxième.



Le tableau `args` reçoit les arguments et `args[0]` désigne le premier.

Après un premier lancement avec le triangle vert à côté de `main`, ouvrez **Run > Edit Configurations...** et sélectionnez la configuration **Main**. Dans le champ des arguments de l'application, saisissez `FirstParam` (1), puis cliquez sur **Run** (2). IntelliJ transmet ce mot à `args[0]` et le programme l'affiche dans le panneau **Run**. Si vous fournissez plusieurs mots séparés par des espaces, ils occupent les positions `args[0]`, `args[1]`, etc. Il faut fournir au moins un argument avant de lancer le code de la capture.



Entrez `FirstParam` dans les arguments de l'application, puis lancez `Main`.